

Rapport sur la thèse de doctorat de Mr Davide Fissore

Matthieu Sozeau
Inria Rennes & LS2N, Université de Nantes

25 août 2026

Chères et chers collègues,

Voici mon rapport sur le mémoire de thèse de doctorat de Mr Davide Fissore, intitulée "Programmation logique pour les preuves interactives : inférence des classes de type et analyse du déterminisme".

Ce mémoire de thèse contribue au domaine de la programmation et la méta-programmation logique dans le contexte des assistants de preuves interactifs. L'auteur étudie en particulier le langage de programmation logique ELPI, implémentant une variante de λ -prolog, ainsi que son plongement dans ROCQ, nommé ROCQ-ELPI. Ses travaux étendent ELPI et ROCQ-ELPI en tirant parti de techniques de programmation logique dans le but d'offrir un meilleur support pour la méta-programmation dans l'assistant de preuve.

Structure du mémoire Après une introduction au contexte de ces travaux (chapitre 2), l'auteur se concentre sur trois aspects principaux : l'implémentation d'un nouveau mécanisme d'inférence des classes de type via ROCQ-ELPI (chapitre 3), le développement d'une solution pour mieux aligner les procédures d'unification entre langage objet et méta-langage (chapitre 4), et l'introduction d'une analyse statique du déterminisme des programmes ELPI (chapitre 5). Ce dernier aspect comprend une présentation formelle et une implémentation efficace de l'analyse pour les programmes ELPI, ainsi qu'une preuve mécanisée de la correction d'une variante simplifiée (chapitre 6). Le mémoire se termine par une conclusion et une discussion sur les perspectives de ces travaux (chapitre 7).

Résolution des classes de type La première contribution de ce mémoire est l'implémentation d'un mécanisme d'inférence des classes de type basée sur ELPI, alternatif à celui de ROCQ. Après avoir présenté les limites d'une représentation naïve des classes et instances, l'auteur propose une intégration profonde et plus naturelle des classes de type dans ROCQ-ELPI : chaque classe de type définie en ROCQ est représentée par un prédicat ELPI correspondant, et chaque instance correspond à une règle pour ce prédicat. Cette interprétation repose sur les outils puissants de méta-programmation offerts par ELPI tels que l'ajout dynamique de prédicats et de clauses. L'auteur montre aussi que les système de priorités et de modes des classes de type de ROCQ peut être transcrit vers leur représentation ELPI, modulo des différences sémantiques bien expliquées. Cette méthode permet de bénéficier directement du moteur de recherche de preuve efficace d'ELPI pour l'inférence des classes. L'utilisateur peut aussi bénéficier de l'expressivité et l'extensibilité supérieures de ROCQ-ELPI pour exprimer des programmes logique plus complexes. L'auteur présente les résultats d'expérimentation de cette implémentation sur des bibliothèques utilisant intensivement les classes. Ces résultats montrent que l'inférence des classes de type s'avère

souvent plus efficace via ELPI que l'inférence native de ROCQ, et ce malgré des coûts de traduction élevés entre ROCQ et ELPI. Ceci s'explique en particulier par l'utilisation côté ELPI d'un mécanisme d'indexation et d'une implémentation efficace du filtrage et de l'unification, pouvant être complétés par l'ajout de règles ad-hoc. L'auteur rapporte enfin les limitations du solveur ELPI, dues notamment à l'absence du dépliage de constantes, qui peut être justifié pour la prédictibilité de la recherche de preuve. Une autre limitation est due à l'encodage des instances qui ne modélise pas correctement les conversions $\beta\eta$, ce qui peut conduire à des échecs d'inférence de classes de type.

Liens pour l'unification La seconde contribution de ce mémoire est l'introduction d'un mécanisme de liens (links) pour traiter ce type de différences entre les algorithmes d'unification de ROCQ et ELPI, liées non seulement aux règles de $\beta\eta$ -conversion mais aussi aux heuristiques d'unification utilisées par ROCQ lorsqu'un problème d'unification n'appartient pas au fragment *pattern*, fragment pour lequel un unificateur le plus général existe. Le mécanisme de lien permet d'abstraire temporairement par des variables d'unification les termes dits problématiques, c'est-à-dire pouvant impliquer une η conversion par exemple, lorsqu'ils apparaissent dans un problème d'unification. L'unification est alors effectuée sur les termes abstraits, et lorsqu'une variable d'unification abstraite est instanciée, le lien associé est examiné, assurant que le problème d'unification de départ est bien traité.

L'auteur développe une justification originale de cette technique dans un cadre relativement simplifié, via une simulation entre un algorithme d'unification pour un langage objet supportant la $\beta\eta$ -conversion et un algorithme d'unification pour un méta-langage ne la supportant pas dans sa théorie équationnelle, mais supportant l'utilisation des liens. Ces deux langages sont représentés dans ROCQ-ELPI, et l'auteur présente cette simulation ainsi qu'une ébauche de preuve montrant que les deux algorithmes d'unification sont équivalents sur des problèmes impliquant $\beta\eta$ -conversion ou sortant temporairement du fragment *pattern*.

Cette partie est la plus complexe et technique du manuscrit. Elle s'appuie sur des procédures de compilation et décompilation pour lier les deux langages et le maintien d'une correspondance entre variables dans chacun d'entre eux. De plus, la gestion des invariants et le flot de contrôle sont non triviaux en présence des différents liens introduits. Cette preuve de simulation étant relativement surprenante et complexe, son exposé est parfois difficile à suivre. Elle bénéficierait de fait d'un traitement plus formel : l'auteur rapporte une tentative de produire une telle preuve dans Abella, arrêtée par des difficultés techniques. Cependant, cette simulation est validée expérimentalement par une large batterie de tests pour une implémentation ELPI usant élégamment des *constraint handling rules*. Les résultats d'expérimentation sur des bibliothèques existantes sont aussi encourageants : ils montrent que ce mécanisme de liens est utile pour traiter une part significative des problèmes d'unification provenant des recherches de classes de type de ROCQ.

Déterminisme des programmes Elpi La troisième contribution de ce mémoire est l'introduction d'un système d'annotation des prédicats et d'une analyse statique permettant d'assurer le déterminisme d'un prédicat ou du résultat d'une requête ELPI, offrant plus de contrôle et de prédictibilité aux utilisateurs. Le déterminisme est une propriété importante pour les programmes logiques, il permet de garantir un comportement fonctionnel pour une large part des prédicats logiques, et donc de faciliter leur compréhension et leur utilisation, en particulier par les utilisateurs de ROCQ plus familiers du paradigme fonctionnel. L'analyse proposée s'appuie sur un système d'annotation des prédicats comme relationnels ou fonctionnels et vérifie la cohérence de ces annotations avec chaque clause d'un programme. De plus cette analyse est flexible : elle supporte les appels incorrects, via une opération d'affaiblissement, permettant de porter du code ELPI existant de façon incrémentale. Techniquement, l'analyse repose sur la politique de

filtrage particulière d'ELPI pour les arguments annotés comme des entrées, permettant d'implémenter un test efficace d'exclusion mutuelle des clauses d'un prédicat, basé sur des arbres de discrimination. Ce test est nécessairement approximatif, mais ne semble pas être une grande limitation en pratique. En effet, l'algorithme est implémenté dans ROCQ-ELPI et déjà expérimenté avec succès sur des bibliothèques non-triviales, validant son utilité : il permet de détecter des bugs de non-déterminisme et de vérifier qu'une large part des programmes ELPI existants sont déterministes.

Preuve de correction de l'analyse du déterminisme L'auteur présente enfin une dernière contribution : une preuve formelle conséquente, en ROCQ, de la correction d'une version simplifiée de l'algorithme d'analyse du déterminisme, justifiant son approche. Le sous-ensemble considéré est un fragment premier ordre du langage avec l'opérateur `cut`, mais sans introduction dynamique de règles ni variables d'ordre supérieur, relativement moins cruciaux pour l'analyse du déterminisme. Cette preuve est basée sur une nouvelle représentation explicite des arbres de recherche avec coupures et une correspondance entre la sémantique opérationnelle originale et une sémantique plus riche reposant sur ces arbres. À partir de cette dernière, l'auteur prouve que l'exécution d'un programme vérifié comme déterministe correspond à un arbre de recherche produisant au plus une solution. Ce résultat est très significatif et démontre une compréhension profonde du fonctionnement d'ELPI et de son moteur d'exécution ainsi qu'une capacité à modéliser et vérifier un algorithme complexe dans ROCQ.

En conclusion, ce mémoire de thèse présente de manière détaillée des contributions multiples, originales et significatives dans le domaine de la programmation logique, en particulier pour les assistants de preuve interactifs. L'auteur a démontré sa maîtrise théorique et technique de ce domaine, incluant la formalisation, la vérification et l'expérimentation des solutions qu'il propose. Ces résultats apportent plus d'efficacité, d'expressivité et de contrôle pour la méta-programmation en ROCQ via ELPI, et plus généralement dans tout cadre de méta-programmation logique. En particulier, en combinant ces résultats, l'auteur a construit une alternative attractive au système d'inférence des classes de type de ROCQ, permettant de bénéficier de l'expressivité et de l'efficacité d'ELPI pour contrôler la recherche d'instances. Ces travaux contribuent d'ores et déjà à une utilisation plus large de ROCQ-ELPI, améliorant le support pour le développement de méta-programmes robustes et prédictibles. Ils ouvrent de nouvelles perspectives pour la méta-programmation dans les assistants de preuve, directement accessible aux utilisateurs.

Pour toutes ces raisons, je donne un avis très favorable à la soutenance de cette thèse par Mr Davide Fissore.

Nantes, le 25 août 2026.
Matthieu Sozeau

